

IME 775 — Lecture 10

Gaussian and Famous Distributions in ML

1. The Gaussian (Normal) Distribution

The most important distribution in machine learning.

1D Gaussian

$$\mathcal{N}(x | \mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

Parameters:

- μ : mean (center of the bell curve)
- σ^2 : variance (width of the bell curve)
- σ : standard deviation

Properties:

- Symmetric about μ
- 68-95-99.7 rule: 68% within $\pm 1\sigma$, 95% within $\pm 2\sigma$, 99.7% within $\pm 3\sigma$
- Peak value: $\frac{1}{\sigma\sqrt{2\pi}}$ at $x = \mu$

Why Gaussian is everywhere:

- Central Limit Theorem: sum of many independent random variables $\rightarrow$ Gaussian
- Maximum entropy distribution for given mean and variance
- Mathematically convenient (closed-form for many operations)

Maximum Entropy — deeper explanation:

Among all continuous distributions with a fixed mean μ and variance σ^2 , the Gaussian uniquely maximizes **differential entropy**:

$$H[p] = - \int p(x) \log p(x) dx$$

This is proven using calculus of variations with Lagrange multipliers — you enforce three constraints ($\int p = 1$, $\int x p = \mu$, $\int x^2 p = \sigma^2 + \mu^2$) and solve for the $p(x)$ that maximizes $H[p]$. The solution is exactly $\mathcal{N}(x \mid \mu, \sigma^2)$.

Why this matters in ML: If you only know the mean and variance of a quantity (e.g., noise in a sensor, residuals in regression), choosing Gaussian is the *least biased* model — it does not sneak in any additional structure beyond what you've measured. Any other distribution with the same mean and variance would implicitly assert extra information you don't have.

*In short: Gaussian is not just convenient — it is the **provably correct** choice under ignorance of higher-order moments.*

Does max entropy = min information?

Yes, but with a careful distinction between two senses of "information":

- **Prior information injected into the model:** Max entropy means minimum. A high-entropy distribution is as spread out as possible — you are not asserting any structure beyond what the constraints (mean, variance) already force. Any lower-entropy distribution with the same mean and variance would implicitly claim to know something extra that concentrates probability somewhere, which you don't.
- **Information gained per observation:** This flips. Shannon information of a single outcome is $I(x) = -\log p(x)$. If $p(x)$ is small (spread-out, high entropy), seeing x is *surprising* — high information content. So a max entropy prior yields more information per observation, because you started knowing less.

	Prior knowledge injected	Surprise per observation
Max entropy (Gaussian)	Minimum	Maximum
Low entropy	Maximum	Minimum

Practical upshot: When you assume Gaussian noise in a model, you are not claiming noise *is* Gaussian — you are claiming you have no reason to believe it has any structure beyond its mean and variance.

Workout: If $X \sim \mathcal{N}(5, 4)$, what are μ and σ ? What is $P(3 \leq X \leq 7)$?

Solution:

- $\mu = 5, \sigma^2 = 4$, so $\sigma = 2$
 - $P(3 \leq X \leq 7) = P(\mu - \sigma \leq X \leq \mu + \sigma) \approx 68\%$ (by the 68-95-99.7 rule)
-

2. Multivariate Gaussian

For $\vec{x} \in \mathbb{R}^d$:

$$\mathcal{N}(\vec{x} \mid \vec{\mu}, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\vec{x} - \vec{\mu})^T \Sigma^{-1} (\vec{x} - \vec{\mu}) \right)$$

Parameters:

- $\vec{\mu} \in \mathbb{R}^d$: mean vector
- $\Sigma \in \mathbb{R}^{d \times d}$: covariance matrix (symmetric, positive definite)

Understanding the Exponent

The expression $(\vec{x} - \vec{\mu})^T \Sigma^{-1} (\vec{x} - \vec{\mu})$ is a **quadratic form** — the Mahalanobis distance!

- Level sets (constant probability) are **ellipses** in 2D
- Axes aligned with **eigenvectors** of Σ

- Axis lengths proportional to **square roots of eigenvalues**

Mahalanobis distance — deeper explanation:

The Mahalanobis distance between a point $\vec{x}$ and a distribution $\mathcal{N}(\vec{\mu}, \Sigma)$ is:

$$D_M(\vec{x}) = \sqrt{(\vec{x} - \vec{\mu})^T \Sigma^{-1} (\vec{x} - \vec{\mu})}$$

Why not just use Euclidean distance? Euclidean distance $\|\vec{x} - \vec{\mu}\|$ treats all dimensions as equally scaled and independent. This breaks in two ways:

- **Scale:** A point 3 units away in a high-variance dimension is unremarkable; the same 3 units in a low-variance dimension is an outlier. Euclidean distance can't distinguish these.
- **Correlation:** If features are correlated (e.g. height and weight), Euclidean distance double-counts shared variance. Σ^{-1} decorrelates the space first, then measures distance.

What Σ^{-1} does geometrically: It stretches dimensions with low variance (making deviations there appear larger) and compresses dimensions with high variance (making deviations there appear smaller). The result is a distance measured in units of *standard deviations*, accounting for the shape of the distribution.

Special case: If $\Sigma = I$ (identity), D_M reduces exactly to Euclidean distance.

Intuition: Mahalanobis distance answers "how many standard deviations is $\vec{x}$ from the mean, in all directions simultaneously?" A point with $D_M = 1$ lies on the 1-SD ellipsoid of the distribution, regardless of orientation or scale.

Special Cases		
Covariance	Shape	Formula
$\Sigma = \sigma^2 I$	Circle (isotropic)	Equal variance in all directions
Σ diagonal	Axis-aligned ellipse	Independent features
Σ general	Rotated ellipse	Correlated features

Workout: For $\Sigma = \begin{pmatrix} 9 & 0 \\ 0 & 1 \end{pmatrix}$, describe the shape of the level sets.

Solution:

- Eigenvalues: $\lambda_1 = 9, \lambda_2 = 1$
 - Eigenvectors: along x_1 and x_2 axes (diagonal matrix)
 - Shape: **axis-aligned ellipse**, stretched along x_1 (semi-axis $\sqrt{9} = 3$) and short along x_2 (semi-axis $\sqrt{1} = 1$)
 - Aspect ratio 3:1
-

3. Connection to PCA and Quadratic Forms

Recall from Chapter 4:

The quadratic form $Q(\vec{x}) = \vec{x}^T A \vec{x}$ defines ellipses when A is positive definite.

Now: The multivariate Gaussian exponent is exactly:

$$Q(\vec{x}) = (\vec{x} - \vec{\mu})^T \Sigma^{-1} (\vec{x} - \vec{\mu})$$

This means:

- **PCA** finds the eigenvectors of $\Sigma \rightarrow$ principal directions of data spread
- **Gaussian** uses $\Sigma^{-1} \rightarrow$ same eigenvectors, reciprocal eigenvalues
- Fitting a Gaussian = estimating $\vec{\mu}$ and Σ from data

Deeper explanation:

Quadratic forms and ellipses. A quadratic form $Q(\vec{x}) = \vec{x}^T A \vec{x}$ with positive definite A traces an ellipse at every constant level $Q = c$. The eigenvectors of A set the orientation; the eigenvalues set the axis lengths. The multivariate Gaussian

exponent is exactly this, centered at $\vec{\mu}$ with $A = \Sigma^{-1}$, so every equal-probability contour of the Gaussian is an ellipse shaped by Σ^{-1} .

Why Σ^{-1} and not Σ ? If Σ has eigenvectors $\vec{v}_i$ with eigenvalues λ_i , then Σ^{-1} has the same eigenvectors but eigenvalues $1/\lambda_i$. A direction with high variance (large λ) gives a small $1/\lambda$ in Σ^{-1} , meaning the Gaussian penalizes deviations there *less* — the ellipse stretches in high-variance directions. This is geometrically correct: being far from $\vec{\mu}$ in a high-variance direction is expected and should not be surprising.

PCA and Gaussian agree on directions, not weights.

	Operates on	High λ direction
PCA	Σ	Large principal component (important)
Gaussian	Σ^{-1}	Slow probability decay (spread out)

Both are consistent — high variance in a direction means data and probability mass are extended in that direction. They are just two views of the same covariance geometry.

Fitting a Gaussian = doing PCA. Given data $\vec{x}_1, \dots, \vec{x}_n$, the maximum likelihood estimates are:

$$\hat{\vec{\mu}} = \frac{1}{n} \sum_i \vec{x}_i, \quad \hat{\Sigma} = \frac{1}{n} \sum_i (\vec{x}_i - \hat{\vec{\mu}})(\vec{x}_i - \hat{\vec{\mu}})^T$$

The sample covariance $\hat{\Sigma}$ is exactly the matrix PCA decomposes. Fitting a Gaussian and running PCA are two interpretations of the same computation — one probabilistic, one geometric.

4. Bernoulli Distribution

Models a single **binary** outcome (coin flip, yes/no, 0/1).

$$\text{Ber}(x \mid \theta) = \theta^x (1 - \theta)^{1-x}, \quad x \in \{0, 1\}$$

Parameters: $\theta \in [0, 1]$ = probability of success

Statistics:

- $E[X] = \theta$
- $\text{Var}(X) = \theta(1 - \theta)$

ML Applications:

- Binary classification output (spam / not spam)
- Logistic regression: $P(y = 1 | \vec{x}) = \sigma(\vec{w}^T \vec{x} + b)$ where σ is the sigmoid function
- Each pixel in binary image generation

Workout: If a spam filter has $P(\text{spam}) = 0.3$, what are $E[X]$ and $\text{Var}(X)$?

Solution:

- $E[X] = \theta = 0.3$
- $\text{Var}(X) = 0.3 \times 0.7 = 0.21$

Deeper explanation:

The formula $\theta^x (1 - \theta)^{1-x}$ is a compact trick to write two cases in one expression. Plug in $x = 1$: $\theta^1 (1 - \theta)^0 = \theta$. Plug in $x = 0$: $\theta^0 (1 - \theta)^1 = 1 - \theta$. That's all it is.

The variance $\theta(1 - \theta)$ has a clean intuition: it is maximized at $\theta = 0.5$ (a fair coin — maximum uncertainty) and hits zero at $\theta = 0$ or $\theta = 1$ (a certain outcome has no randomness). The more certain you are, the less variance.

In logistic regression, the network's final sigmoid output literally *is* the parameter θ of a Bernoulli — the model is saying "I believe there is probability θ that this example is class 1", and the Bernoulli is what describes that belief formally.

5. Binomial Distribution

Sum of n independent Bernoulli trials:

$$\text{Bin}(k \mid n, \theta) = \binom{n}{k} \theta^k (1 - \theta)^{n-k}$$

Parameters: n = number of trials, θ = success probability

Statistics:

- $E[X] = n\theta$
- $\text{Var}(X) = n\theta(1 - \theta)$

Connection: If $X_i \sim \text{Ber}(\theta)$ independently, then $\sum_{i=1}^n X_i \sim \text{Bin}(n, \theta)$

Deeper explanation:

The formula $\binom{n}{k} \theta^k (1 - \theta)^{n-k}$ has three separable parts that each do something concrete:

- θ^k : probability of getting exactly k successes in some fixed order
- $(1 - \theta)^{n-k}$: probability of the remaining $n - k$ failures
- $\binom{n}{k}$: number of different orderings in which those k successes could have appeared across n trials

Multiply them together and you get the total probability of exactly k successes regardless of order. Intuition: you count all arrangements that produce the desired count, then weight by how likely any one of them is.

A useful hidden connection: as $n \rightarrow \infty$, the Binomial converges to a Gaussian (this is one direct instance of the CLT). So even discrete counting distributions eventually look bell-shaped at scale — another reason Gaussians appear everywhere.

6. Categorical Distribution

Generalization of Bernoulli to K categories.

$$\text{Cat}(x \mid \vec{\theta}) = \prod_{k=1}^K \theta_k^{[x=k]}$$

where $[x = k]$ is 1 if $x = k$, else 0.

Constraint: $\sum_{k=1}^K \theta_k = 1$

One-hot encoding: Represent category k as vector $\vec{e}_k$ with 1 at position k :

$$P(\vec{x} = \vec{e}_k) = \theta_k$$

ML Applications:

- Multi-class classification: $P(y = k \mid \vec{x}) = \text{softmax}_k(\vec{z})$
- Language models: next-word prediction over vocabulary
- Topic models: document-topic assignment

Workout: A classifier outputs $\vec{\theta} = (0.7, 0.2, 0.1)$ for classes (cat, dog, bird). What is $P(\text{dog})$?

Solution: $P(\text{dog}) = \theta_2 = 0.2$

The predicted class is **cat** (highest probability: 0.7).

Deeper explanation:

The formula $\prod_{k=1}^K \theta_k^{[x=k]}$ looks dense but collapses trivially. The indicator $[x = k]$ is 1 for exactly one k (the true class) and 0 for all others. So $\theta_k^1 = \theta_k$ for the true class and $\theta_k^0 = 1$ for every other class — the entire product reduces to just $\theta_{\text{true class}}$. It is an elegant way to select one value from a vector using only multiplication.

One-hot encoding is the vector version of this: instead of writing "the answer is class 3", you write $[0, 0, 1, 0, \dots]$. Every position is 0 except the true class. This is the standard format neural networks use to consume labels and produce outputs.

Softmax is how you *produce* a valid Categorical distribution from raw network scores (logits). It converts arbitrary real numbers into probabilities that sum to 1 — exactly the $\vec{\theta}$ vector the Categorical needs. Without softmax, there is no guarantee the network's outputs are non-negative or sum to 1.

7. Multinomial Distribution

Generalization of Binomial to K **categories** over n trials.

$$\text{Mult}(\vec{m} \mid n, \vec{\theta}) = \frac{n!}{m_1! m_2! \dots m_K!} \prod_{k=1}^K \theta_k^{m_k}$$

where m_k = count of category k , $\sum_k m_k = n$

Statistics:

- $E[m_k] = n\theta_k$

- $\text{Var}(m_k) = n\theta_k(1 - \theta_k)$

ML Application: Bag of Words

A document with n words drawn from vocabulary of K words:

$$P(\text{document}) = \text{Mult}(\vec{m} \mid n, \vec{\theta})$$

where m_k = count of word k , θ_k = probability of word k .

Workout: In 100 emails, a word appears with $\theta = 0.05$. What is the expected count and variance?

Solution:

- $E[m] = n\theta = 100 \times 0.05 = 5$
- $\text{Var}(m) = n\theta(1 - \theta) = 100 \times 0.05 \times 0.95 = 4.75$

Deeper explanation:

The formula $\frac{n!}{m_1! \cdots m_K!} \prod_k \theta_k^{m_k}$ has two separable parts:

- $\prod_k \theta_k^{m_k}$: probability of one specific sequence with those exact word counts in a fixed order
- $\frac{n!}{m_1! \cdots m_K!}$: the multinomial coefficient — how many different orderings of n words produce those same counts

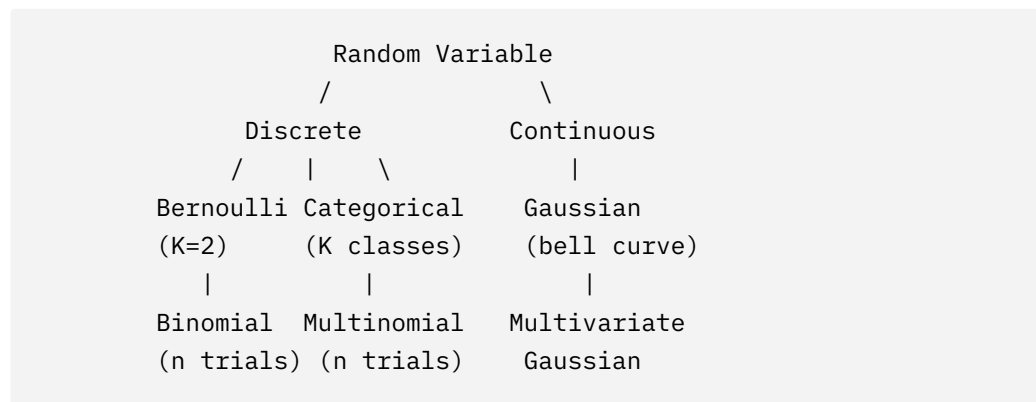
Multiplied together, they give the total probability of a document having word count vector $\vec{m}$, regardless of word order. This is exactly the Bag of Words assumption: word *order* is ignored, only *counts* matter.

The family structure tying all four sections together:

	Single trial	n trials
2 outcomes	Bernoulli	Binomial
K outcomes	Categorical	Multinomial

Each entry is the one to its left repeated n times, or the one above it generalized to more outcomes. Bernoulli and Categorical differ only in number of classes; Bernoulli and Binomial differ only in number of trials. Multinomial is both generalizations applied simultaneously.

8. Distribution Family Tree



Summary Table

Distribution	Domain	Parameters	$E[X]$	$\text{Var}(X)$	ML Use
Bernoulli	$\{0, 1\}$	θ	θ	$\theta(1 - \theta)$	Binary classification
Binomial	$\{0, \dots, n\}$	n, θ	$n\theta$	$n\theta(1 - \theta)$	Count data
Categorical	$\{1, \dots, K\}$	$\vec{\theta}$	—	—	Multi-class output
Multinomial	counts	$n, \vec{\theta}$	$n\theta_k$	$n\theta_k(1 - \theta_k)$	Bag of words

Gaussian	$\mathbb{R}$	μ, σ^2	μ	σ^2	Regression, latent
Multivariate Gaussian	$\mathbb{R}^d$	$\vec{\mu}, \Sigma$	$\vec{\mu}$	Σ	Generative models

9. From Distributions to ML Algorithms

ML Algorithm	Distribution Used	How
Logistic Regression	Bernoulli	$P(y = 1 \vec{x})$ modeled as Bernoulli
Softmax Classifier	Categorical	$P(y = k \vec{x})$ via softmax
Naive Bayes	Multinomial	Word counts in documents
Gaussian Mixture	Multivariate Gaussian	Cluster data with K Gaussians
VAE	Gaussian	Latent space $\vec{z} \sim \mathcal{N}(\vec{0}, I)$
Linear Regression	Gaussian	Noise assumed Gaussian

Key Takeaways

1. **Gaussian** is the workhorse: noise modeling, latent spaces, maximum entropy
2. **Bernoulli → Categorical**: binary to multi-class classification
3. **Binomial → Multinomial**: counting across categories (bag of words)
4. **Covariance matrix** Σ connects probability to PCA/SVD from Chapter 4
5. These distributions are the building blocks for Chapter 6 (Bayesian methods)

PyTorch Connection

```
import torch
import torch.distributions as D

# Gaussian
normal = D.Normal(loc=0.0, scale=1.0)
samples = normal.sample((1000,))
log_prob = normal.log_prob(torch.tensor(0.5))

# Multivariate Gaussian
mu = torch.tensor([1.0, 2.0])
cov = torch.tensor([[2.0, 0.5], [0.5, 1.0]])
mvn = D.MultivariateNormal(mu, cov)
samples_2d = mvn.sample((500,))

# Bernoulli
bern = D.Bernoulli(probs=0.7)
flips = bern.sample((100,))
print(f"Heads: {flips.sum()}") # ≈ 70

# Categorical (e.g., softmax output)
logits = torch.tensor([2.0, 1.0, 0.5])
cat = D.Categorical(logits=logits)
classes = cat.sample((100,))

# Multinomial
probs = torch.tensor([0.5, 0.3, 0.2])
multi = D.Multinomial(total_count=100, probs=probs)
counts = multi.sample()
print(f"Counts: {counts}") # ≈ [50, 30, 20]
```